

Redis

1. What is Redis?

Redis is an in-memory data store, used as a cache, a database and a message broker. It keeps data in memory and answers in well under a millisecond.

The name is short for Remote Dictionary Server, and that describes it: a dictionary that other machines can reach over the network.

It is key-value, but the values are data structures. A value can be a string, a list, a set, a hash or a sorted set, and there are commands that operate on them in place. That is the difference from a plain cache.

2. Why use Redis?

Problem	What Redis does
The database is the bottleneck on reads	Serve the answer from memory
A session must be shared across instances	Keep it in one place every instance reaches
Something is counted on every request	<code>INCR</code> , with no read-modify-write
Requests need rate limiting	A counter with an expiry
Two services need a queue	Lists, or streams
Several instances must not act at once	A lock, from a single atomic <code>SET</code>

The speed comes from memory and simplicity, not from cleverness. Data is in RAM, the protocol is small, and one command runs at a time.

3. Redis vs a traditional database

	Redis	Relational database
Data lives in	Memory, optionally saved to disk	Disk
Latency	Under a millisecond	Single-digit milliseconds
Finds data by	The exact key	Any column, joined across tables
Schema	None	Defined and enforced
Transactions	Limited. No rollback on a failed command	Full ACID
Size limit	The available memory	The available disk

They answer different questions. Redis answers "give me this key" very fast. A relational database answers "which rows match this" at all. A system with both is normal; a system that replaced one with the other usually regrets it.

4. In-memory data store

In-memory means the data lives in RAM rather than on disk. That is where the speed comes from, and it brings two costs.

Memory is limited. The dataset has to fit, which is why Redis has a memory ceiling and a policy for what to remove when it is reached.

```
maxmemory 4gb
maxmemory-policy allkeys-lru
```

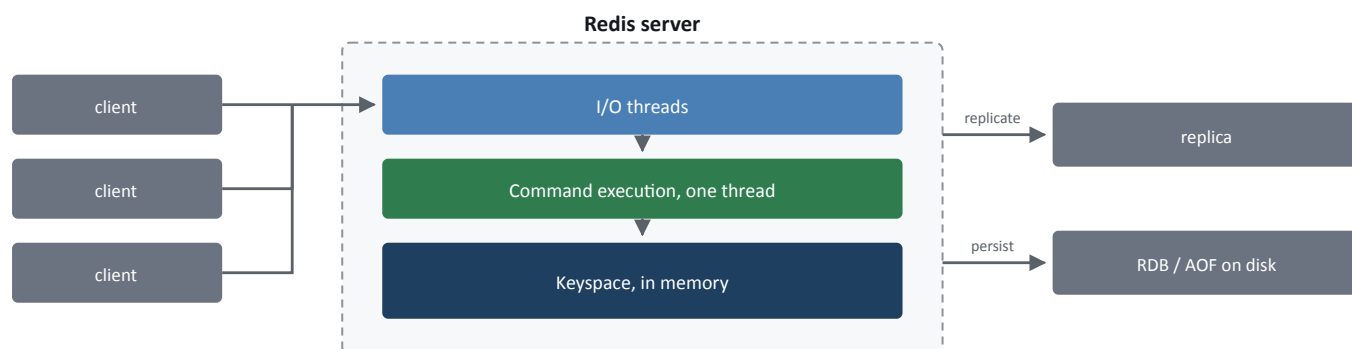
Memory is volatile. A restart loses everything unless the data was written somewhere, and Redis offers two ways to do that:

	RDB	AOF
Writes	A point-in-time snapshot	Every write command, appended
On restart	Loads the last snapshot	Replays the log
Loses	Everything since the last snapshot	A second at most
Cost	Small	Larger files, slower restart

Redis is durable enough for a cache and not for a ledger. Even with the append-only file, the default flushes once a second, so a crash can lose that second. Data that must not be lost belongs in a database, with Redis in front of it.

5. Redis architecture

One server process holds the data and answers clients over a TCP port, 6379 by default.



Many clients, many I/O threads, and one thread executing commands.

Beyond one server there are three optional pieces, each solving one problem:

Piece	Solves
Replication	Read scaling, and a copy of the data
Sentinel	Automatic failover when the primary dies
Cluster	Splitting a dataset larger than one machine

6. Key-value model

Every value is found by its exact key. There is no way to search by value.

Keys are plain strings, and the convention is colon-separated namespaces:

```
user:41:name  
user:41:session  
order:9012:status
```

No querying by value is the design constraint. Anything that needs to be looked up must be a key, or must have a second key built for it. That is why Redis data modeling is mostly deciding what the keys are.

The values are where Redis differs from a plain cache, and that is the next section.

7. Redis data types

Type	Holds	Typical use
String	Bytes, up to 512 MB	Cached values, counters
Hash	Field-value pairs	One object, updated field by field
List	An ordered sequence, duplicates allowed	Queues, recent items
Set	Unique members, unordered	Membership, tags
Sorted set	Unique members, each with a score	Leaderboards, ranges by score
Stream	An append-only log with ids	Event streams, consumer groups
Bitmap	Bits inside a string	One flag per user, at scale
HyperLogLog	An approximate distinct count	Unique visitors, cheaply
Geospatial	Coordinates, in a sorted set	What is near this point

The last three are less familiar and each solves one problem very well.

```

SETBIT active:2026-09-16 41 1      user 41 was active today
BITCOUNT active:2026-09-16        how many users were

PFADD visitors:2026-09-16 "u41"     record a visitor
PFCOUNT visitors:2026-09-16        roughly how many distinct ones

GEOADD stores 12.49 41.89 "rome"     add a location
GEOSEARCH stores FROMLONLAT 12.5 41.9 BYRADIUS 5 km ASC

```

A bitmap is not a separate type. It is a string treated as a row of bits, so one bit per user per day tracks activity for a million users in 125 kilobytes.

HyperLogLog trades accuracy for memory. It counts distinct items in a fixed 12 kilobytes with roughly one percent error, however many items there are. Use it where the exact number does not matter.

Geospatial is a sorted set underneath, with the score encoding the coordinates. That is why radius searches are fast and why the values behave like set members.

8. Redis commands

```

SET user:41:name "Rohit"
OK
GET user:41:name
"Rohit"
EXPIRE user:41:name 1800
(integer) 1
TTL user:41:name
(integer) 1800
INCR page:views
(integer) 1

```

Command	Does
SET / GET / DEL / EXISTS	Write, read, remove, and test a key
EXPIRE / TTL	Set and read a lifetime, in seconds
INCR / DECR	Atomic counters
HSET / HGET	Fields of a hash
LPUSH / RPOP	Both ends of a list
SADD / SISMEMBER	Set membership
ZADD / ZRANGE	Sorted sets, by score
SCAN	Walk the keyspace in batches, safely

Never run `KEYS` on a production server. It scans every key in one operation and, because commands are single-threaded, nothing else runs until it finishes. `SCAN` does the same work in small batches and lets other commands through.

`EXPIRE` is what makes Redis a cache. Without a lifetime, a key stays until something deletes it, and the memory ceiling arrives sooner than expected.

9. Redis client

The client is the library in our application that speaks the Redis protocol over TCP. In Java the choices are Jedis and Lettuce, and Spring Boot uses Lettuce by default.

```
@Autowired
StringRedisTemplate redis;

public void cacheName(long userId, String name) {
    redis.opsForValue().set("user:" + userId + ":name", name,
        Duration.ofMinutes(30));
}

public String readName(long userId) {
    return redis.opsForValue().get("user:" + userId + ":name");
}
```

The client does the work Redis does not. Connection pooling, retries, and turning objects into bytes are all the library's job. Redis stores bytes and has no idea what they mean.

10. Redis server

The server is the process that holds the keyspace and executes commands.

	Detail
Port	6379 by default
Configuration	<code>redis.conf</code> , or <code>CONFIG SET</code> at run time
Statistics	<code>INFO</code> , which covers memory, clients, persistence and more
Scaling up	More instances, not more threads

One Redis process uses roughly one core for commands. A machine with sixteen cores runs sixteen Redis instances rather than one large one, which is also how Redis Cluster shards a dataset.

11. Redis databases and SELECT

One Redis server holds sixteen numbered logical databases, 0 to 15. A connection picks one with `SELECT`, and starts on 0.

```
SELECT 1
OK
SET key "value"
OK
FLUSHDB      clears database 1 only
FLUSHALL     clears all sixteen
```

They are not isolation. All sixteen share one process, one memory limit and one thread, so a slow command in database 3 blocks database 0 as well.

Do not use them to separate applications or tenants. Redis Cluster does not support them at all, so anything built on them cannot be sharded later. Key prefixes do the same job and survive the move.

12. The single-threaded execution model

Redis executes one command at a time, in one thread.

That sounds like a limitation and is mostly the reason it is fast:

Because there is one thread	Result
No locks are needed	No time is spent acquiring them
No context switching between commands	Predictable, low latency
Two commands never interleave	Every command is atomic for free

Atomicity is free, and this is the part worth saying out loud. `INCR` cannot lose an update, and `SET key value NX` either succeeds or does not, which is why it works as a lock. Neither needs a transaction.

Blocking is total. The same property means one slow command stops everything: `KEYS` on a large keyspace, a `DEL` of a huge collection, a long Lua script. There is no second thread to serve anyone while it runs.

The usual bottleneck is the network, not the CPU. A single thread handles well over a hundred thousand simple commands a second, which is more than most applications ask of it.

13. I/O threading in Redis 7

Redis 7 can use several threads for reading from and writing to sockets, while command execution stays on one thread.

Stage	Threads
Reading the request from the socket	Several
Parsing and executing the command	One, always
Writing the reply to the socket	Several

This does not make Redis multi-threaded. Commands still run one at a time, so none of section 12 changes: atomicity is still free, and a slow command still blocks everything.

It helps when the socket work is the bottleneck, which means many connected clients or large values moving in and out. It does nothing for a workload that is slow because the commands themselves are slow.

Leave it off unless a measurement says otherwise. The default is one I/O thread, and most deployments never need more.

14. Why Redis is used as a cache

A cache sits in front of something slower. Redis suits the job because reads take under a millisecond, keys can expire on their own, and every instance of the service reaches the same copy.

Five problems come with caching, and they have names because they all happen.

Problem	What happens	The usual fix
Invalidation	The cache still holds what the database no longer says	A TTL, plus deleting the key on write
Warming	A cold cache after a restart sends everything to the database	Load the hot keys before taking traffic
Stampede	A hot key expires and every request misses at once	One request refills while the others wait
Penetration	Requests for keys that exist nowhere, so nothing is ever cached	Cache the "not found", or a bloom filter
Avalanche	Many keys expire at the same moment	Add jitter to the TTL

Invalidation is the hard one because there is no right answer. A short TTL means more misses; a long one means more staleness; deleting on write means the delete has to succeed. Every option is a trade, which is why the phrase about it being one of the two hard problems in computing exists.

Penetration is the one that gets attacked. A request for `user:99999999` misses the cache, misses the database, and caches nothing, so repeating it is a free way to put load on the database. Caching the empty result with a short TTL closes it.

Avalanche is usually self-inflicted. Warming a thousand keys at startup with the same thirty minute TTL means a thousand keys expire in the same second. Spreading the TTL over a range removes it entirely.

```
Duration ttl = Duration.ofMinutes(30).plusSeconds(random.nextInt(300));
```

15. The cache-aside pattern

In cache-aside the application owns the cache. It looks there first, and on a miss it reads the database, writes the answer into the cache, and returns it.

```
public Profile getProfile(long userId) {
    String key = "user:" + userId + ":profile";

    Profile cached = cache.get(key);
    if (cached != null) {
        return cached;
    }

    Profile profile = repository.findById(userId).orElseThrow();
    cache.put(key, profile, ttlWithJitter());
    return profile;
}
```

It is the default pattern because it is simple, only the data actually requested is cached, and Redis being down degrades the service to slow rather than broken.

Five things go wrong with it, and they are worth knowing by name.

Race conditions. Two requests miss together, both read the database, both write. Usually harmless because the value is the same. The damaging version involves a write: a reader reads the old row, a writer updates the row and deletes the key, and then the reader writes its stale value into the empty cache. The cache is now wrong until the TTL expires.

Stale data. After a write, the cache still holds the previous value. Delete the key on write rather than updating it, because two concurrent writers can update a cache out of order and leave the loser's value in place. A deleted key is simply refetched.

Cache stampede. For a hot key, a single expiry sends every concurrent request to the database at once. A short lock around the refill lets one request do the work:


```

if (cache.setIfAbsent(key + ":lock", "1", Duration.ofSeconds(10))) {
    // this request refills
} else {
    // the others wait briefly, or serve the stale value
}

```

Database overload. The general case: a cold cache after a deploy, or many keys expiring together, sends far more traffic to the database than it normally sees. Warming, jittered expiry and a cap on concurrent refills are the three answers, and they are the same three from the previous section.

Failure between the database and Redis. The database commit succeeds and the cache delete fails, or the reverse. There is no transaction spanning the two, so this cannot be prevented, only bounded: delete after the commit, retry the delete, and rely on the TTL as the backstop.

The TTL is the safety net under all five. Every mitigation above can fail, and a lifetime on the key turns any of these from a permanent wrong answer into a temporary one. A cache entry with no expiry is a bug waiting for the right sequence of events.

The pieces together

Piece	What it is
In-memory	Why it is fast, and why it can lose data
Key-value	Found by exact key, never searched by value
Data structures	Lists, sets, hashes and sorted sets, not just strings
EXPIRE	What makes it a cache rather than a store
Single thread	Why every command is atomic, and why one slow one hurts
I/O threads	Parallel sockets, serial commands
SELECT	Sixteen namespaces that are not isolation
Replication, Sentinel, Cluster	Copies, failover and sharding
Bitmap, HyperLogLog, Geo	Three types that trade exactness for size
Cache-aside	Look in the cache, fall back to the database, fill it
Stampede, penetration, avalanche	The three ways a cache sends load downstream

For example, a service that reads a user profile on every request. The profile is cached under `user:41:profile` with a thirty minute expiry, so almost every request is answered from memory and the database sees a fraction of the load. A counter under `ratelimit:41` is incremented with `INCR` and expires after a minute, and because commands are atomic no two instances can miscount it. One night somebody runs `KEYS user:*` to see how many are cached, and every request in the service times out until it finishes.

The idea that explains most Redis behavior is the single thread. Speed, atomicity, and the one way to bring a server down all come from the same place.